Arend Proof Assistant

Valery Isaev

December 19, 2019

Basic Theory

Universes

Higher Inductive Types

Class System

Future Plans

The Interval Type

- Arend is similar to CTT in some ways.
- It has the interval type, path types, and higher inductive types.
- The interval type is the unit type with two constructors:

$$\frac{}{\Gamma \vdash \mathrm{I}} \qquad \frac{}{\Gamma \vdash \mathrm{left} : \mathrm{I}} \qquad \frac{}{\Gamma \vdash \mathrm{right} : \mathrm{I}}$$

$$\frac{\Gamma, x : \mathrm{I} \vdash A \qquad \Gamma \vdash a : A[\mathrm{left}/x] \qquad \Gamma \vdash i : \mathrm{I}}{\Gamma \vdash \mathrm{coe}(x.A, a, i) : A[i/x]}$$

- In particular, the interval type is contractible.

Path Types

- If $x : I \vdash A$, then a path between $a : A[\mathrm{left}]$ and $a' : A[\mathrm{right}]$ is simply a function $f : \prod_{i:\mathrm{I}} A$ such that $f\ \mathrm{left} \equiv a$ and $f\ \mathrm{right} \equiv a'$.
- Thus, the path type can be defined as the type of such functions:

$$\frac{\Gamma, x : I \vdash A \qquad \Gamma \vdash a : A[\mathrm{left}] \qquad \Gamma \vdash a' : A[\mathrm{right}]}{\Gamma \vdash \mathrm{Path}(x.A, a, a')}$$

$$\frac{\Gamma, x : I \vdash f : A}{\Gamma \vdash \mathrm{path}(x.f) : \mathrm{Path}(x.A, f[\mathrm{left}/x, f[\mathrm{right}/x]])}$$

$$\frac{\Gamma \vdash p : \mathrm{Path}(x.A, a, a') \qquad \Gamma \vdash i : \mathrm{I}}{\Gamma \vdash p@i : A[i/x]}$$

Properties of Path Types

- If $x \notin \mathrm{FV}(A)$, we will denote $\mathrm{Path}(x.A, a, a')$ by $a =_A a'$ or $a = a'$.
- Cubical fillers can be defined by coercing elements along path types.
- If $p : a = b$, $q : a = a'$, and $p' : a' = b'$, then we can define a 2-dimensional filler as follows:

$$\mathrm{coe}(x.\, p@x = p'@x, q, i) : p@i = p'@i$$

- Transport can be defined as follows:

$$\mathrm{transport}(x.\, B, p, b) \equiv \mathrm{coe}(y.\, B[p@y/x], b, \mathrm{right})$$

J operator

- The J operator also can be defined.
- Arend also has pattern matching on the reflexivity proof.

Functional Extensionality

- Functional extensionality holds in this theory. If $p : \prod_{x:A} f\,x = g\,x$, then

$$\mathrm{path}(ix.\, p\,x@i) : f = g$$

- The type $\prod_{x:A} f\,x = g\,x$ is *isomorphic* to $f = g$.

Comparison with CTT

- Arend's theory has simpler rules.
- It does not have canonicity property.

Univalence

- Univalence can be defined for all types, i.e., we do not have to assume the existence of universes.
- If $\Gamma \vdash f : A \to B$, $\Gamma \vdash g : B \to A$, and p and q are proofs that f and g are inverses of each other, then

$$\frac{\Gamma \vdash i : I}{\Gamma \vdash \mathrm{iso}(A, B, f, g, p, q, i)}$$

$$\begin{aligned} \mathrm{iso}(A, B, f, g, p, q, \mathrm{left}) &\equiv A \\ \mathrm{iso}(A, B, f, g, p, q, \mathrm{right}) &\equiv B \\ \mathrm{coe}(x.\, \mathrm{iso}(A, B, f, g, p, q, x), a, \mathrm{right}) &\equiv f\ a \end{aligned}$$

- If a universe is closed under iso, then it is univalent in the usual sense.

Basic Theory

Universes

Higher Inductive Types

Class System

Future Plans

Level Inference

- Arend has cumulative universes.
- It is rarely required to specify the levels explicitly:

```
\func id {A : \Type} (a : A) => a
\data Sum (A B : \Type) | inl A | inr B
\func type-in-type : \Type => \Type
```

- Every definition has exactly one level parameter.
- When a definition is invoked, the level parameter is inferred automatically.

Homotopy Level

- Universes in Arend are actually parameterized by *two* levels: the predicative level and the homotopy level.
- A universe of a given homotopy level is closed under most of constructions on types: sum-types, Σ-types, Π-types, path types, ...
- This is a conservative extension. It can be proved that the universe of n-types is equivalent to a subuniverse of ∞-types.
- Every type in the universe of n-types has the correct homotopy level thanks to the axiom

$$\textbf{func}\ \mathrm{pathInProp}\{A : \textbf{Prop}\}(aa' : A) : a = a'$$

- If some data type is *provably* an n-type, it can be put in the universe of n-types.

Propositions

- The universe of propositions is impredicative.
- This implies that Arend has resizing axioms.
- A function can be declared as a lemma.
  - The type of a lemma must be a proposition.
  - Lemmas do not evaluate.

Basic Theory

Universes

Higher Inductive Types

Class System

Future Plans

Defining HITs

- Every constructor of a data type can be endowed with computational rules just as functions.
- For example, S^1 can be defined as a data type with two constructors $\mathrm{base} : S^1$ and $\mathrm{loop} : \mathrm{I} \to S^1$ such that the following conditions hold:

$$\mathrm{loop}\,\mathrm{left} \equiv \mathrm{base}$$

$$\mathrm{loop}\,\mathrm{right} \equiv \mathrm{base}$$

- In Arend, this can be written as follows:

```
\data S1
  | base
  | loop I \with {
    | left => base
    | right => base
  }
```

Truncated Data Types

- If a data type is marked as truncated, it will be put in the specified universe.
- For example, the propositional truncation can be defined as follows:

```
\truncated \data Trunc (A : \Type) : \Prop
  | in A
```

- Such a data type can be eliminated only into types living in the same universe.

Pattern Matching on HITs

- Functions over HITs can be defined as usual by pattern matching with a side condition that they respect computational rules for constructors.
- If the type of a function is an n-type, then some higher constructors can be omitted in the definition of this function.
- For example, we can omit the loop constructor in the following function since its codomain is a set:

```
\func decode_encode (x : Sphere1) (c : code x)
  : encode x (decode x c) = c \elim x
  | base1 => encode_wind c
```

Quotient sets

- Quotient sets can be defined as truncated higher inductive types.
- To define a function of the form $A/\sim \to A/\sim \to A/\sim$, it is only necessary to consider 3 cases, the fourth one can be omitted.

Basic Theory

Universes

Higher Inductive Types

Class System

Future Plans

Inheritance

- A class is a Σ-type with named fields.
- Classes can be inherited. A subclass of C is also a subtype of C.
- A subclass can add new fields or implement previously defined fields.

Class Types

- Every class is also a Haskell-style class type.
- Instances of a class can be defined as ordinary functions.
- Instance inferrence mechanism searches for an appropriate instance.
- Class types interact well with inheritance.

Sections

- Classes can be used to put a set of common hypothesis into a single place. This is similar to Coq's sections.
- Such sections can be inherited and passed as arguments to functions.
- Instances of sections can be manipulated as ordinary data.

Anonymous Extensions

- If we want to define some structure, for example, Ring, then we can put everything inside the class or we can put some pieces of the data in its parameters.
- Which option should we choose? Sometimes it is desirable to have all the options at the same time.
- Arend's classes solve this problem with anonymous extensions.
- An anonymous extension is an expression that consists of a class name and a set of implementations of some of its fields.

Example

- Monoid is the type of monoids.
- $\mathrm{Monoid}\ \mathrm{Int}$ is the type of monoid structures on Int.
- $\mathrm{Monoid}\ \mathrm{Int}\ 0\ +$ is the type of proofs that 0 and $+$ determine a monoid structure on Int.

Basic Theory

Universes

Higher Inductive Types

Class System

Future Plans

- Some form of automatization.
- Universes for other kinds of objects:
  - Pointed types
  - Spectra
  - ∞-categories

  The first two should be a conservative extension.

arend-lang.github.io